Study Session Tracker

1. System Overview

The Study Session Tracker is a full-stack web application designed to help students manage and analyze their study habits. Users can create accounts, add courses, and log study sessions including duration, topic, and focus level. The application also provides basic analytics such as total study time and study time per course, helping users better understand their productivity.

The system follows a client-server architecture, where a React frontend communicates with a Node.js/Express backend, which interacts with a MongoDB database.

2. Database Design

The database consists of three main collections:

Users

Stores student information:

- name
- username
- email
- program

Courses

Stores course-related data:

- courseName
- difficulty (1–5)
- hasExam
- hasSeminar

StudySessions

Stores study session details:

- userId (reference to Users)
- course (reference to Courses)
- date
- durationMinutes
- topic
- focusLevel
- notes

Relationships

Each study session references:

a user through `userId`
a course through `course`

These relationships are implemented using MongoDB ObjectId references and populated using Mongoose `.populate()`.

3. API Endpoints

Endpoint 1: Create Study Session

Route:
POST `/api/study-sessions`

Request Body Example:

```
{
  "userId": "USER_ID",
  "course": "COURSE_ID",
  "date": "2026-04-27",
  "durationMinutes": 60,
```

```
  "topic": "Probability distributions",
  "focusLevel": 4
}
```

Response Example:

```
{
  "_id": "SESSION_ID",
  "durationMinutes": 60,
  "topic": "Probability distributions",
  "focusLevel": 4
}
```

Endpoint 2: Total Study Time by Course (Custom)

Route:
GET `/api/study-sessions/stats/total-by-course`

Description:
Returns total study minutes grouped by course using MongoDB aggregation.

Response Example:

```
[
  {
    "courseName": "Mathematical Statistics",
    "totalMinutes": 180
  }
]
```

4. Reflection

One of the main challenges was handling relationships between users, courses, and study sessions. At first, managing ObjectId references and ensuring correct data flow between frontend and backend caused errors, especially when sending incorrect field names or data types.

This was solved by carefully aligning the frontend request structure with the backend schema and adding validation in Mongoose. Another challenge was structuring the frontend code properly. Initially, all logic was inside a single App.jsx file, which made it difficult to manage and scale.

This was improved by refactoring the application into separate components (UserSection, CourseSection, SessionSection), making the code more modular and easier to maintain.

5. Feature Iteration

One feature that evolved during development was the study session management system.

Initially:

- Study sessions could only be created and displayed.

Later improvements:

- Added filtering by user
- Implemented total study time calculation
- Added a simple chart visualization
- Implemented update (edit) functionality
- Added delete functionality

This progression can be seen in the Git history, where functionality was gradually expanded and improved through multiple commits.

6. Conclusion

The project successfully demonstrates a full-stack application with CRUD operations, data relationships, and basic analytics. It highlights the importance of proper data modeling, frontend-backend integration, and incremental development using Git.

If further developed, the application could include authentication, advanced data visualization, and improved UI responsiveness.